

CO4_AT1 – DATA INTERPRETATION: ANSWERS

1. Real-Time Detection System

Method	Accuracy	Speed
Viola-Jones	78%	Fast
YOLO	92%	Moderate
Deep Learning	95%	Slow

Comparison

- Viola-Jones: Lowest accuracy (78%) but the fastest processing speed. It uses simple Haar-like features and cascaded classifiers, making it computationally lightweight but less accurate under complex conditions.
- YOLO (You Only Look Once): Achieves a strong balance with 92% accuracy and moderate speed. It performs detection in a single forward pass of the network, making it suitable for near real-time applications.
- Deep Learning (e.g., Faster R-CNN/CNN-based models): Highest accuracy (95%) but slowest, since it involves deeper networks and multiple processing stages (region proposal + classification).

Evaluation and Decision

For a surveillance system operating under strict latency constraints while maintaining acceptable accuracy, YOLO is the most suitable choice.

- Latency requirement: Strict latency rules out Deep Learning, since 'Slow' processing cannot support real-time frame-by-frame surveillance.
- Accuracy requirement: Viola-Jones, while fast, has an accuracy of only 78%, which is too low for reliable surveillance (higher risk of missed detections/false alarms).
- Balanced trade-off: YOLO offers 92% accuracy at moderate speed, which is close to real-time and accurate enough for practical surveillance use cases such as intrusion detection or crowd monitoring.

Justification (Deployment Conditions)

Surveillance systems typically run on edge devices or embedded hardware with limited computational resources, and they must process continuous video streams. YOLO's single-pass architecture is optimized for exactly this scenario — it delivers high-enough accuracy without the heavy computational cost of full deep learning pipelines, making it deployable on real-time video feeds without significant frame drops.

2. Robust Object Detection

Method	Normal	Occlusion	Low Light
Viola-Jones	80%	60%	55%
YOLO	90%	75%	70%
Deep Learning	95%	85%	80%

Comparison of Robustness

- Viola-Jones: Performance drops sharply under occlusion (60%) and low light (55%). It relies on fixed Haar features and edge/intensity patterns, which are highly sensitive to lighting changes and partial visibility.
- YOLO: More robust, maintaining 75% under occlusion and 70% in low light. Its learned convolutional features generalize better than handcrafted features, though performance still degrades under difficult conditions.
- Deep Learning: Most robust across all conditions (85% occlusion, 80% low light) due to deeper feature hierarchies that can learn complex, invariant representations from large training datasets.

Evaluation

Deep Learning provides the best overall performance across normal, occluded, and low-light conditions, showing the smallest performance drop (95% → 80%) compared to Viola-Jones (80% → 55%) and YOLO (90% → 70%).

Trade-offs

- Deep Learning: Best robustness and accuracy, but computationally expensive and slow — not ideal where real-time response is critical.
- YOLO: Good balance of robustness and speed — a practical middle ground when both accuracy and real-time performance matter.
- Viola-Jones: Fast and lightweight but unreliable in occlusion or poor lighting — suitable only for controlled, well-lit environments with minimal occlusion.

Overall, the choice depends on priority: Deep Learning for accuracy-critical, offline or high-resource applications; YOLO for real-time systems needing reasonable robustness; Viola-Jones only for simple, constrained, resource-limited settings.

3. Optical Flow Analysis

Comparison

- Method A: Produces stable, smooth motion vectors but fails to capture small or subtle motions. This typically corresponds to approaches using larger aggregation windows or regularization (e.g., Lucas-Kanade with large windows, or Horn-Schunck with heavy smoothing), which suppress noise at the cost of sensitivity.
- Method B: Detects fine, small-scale motion accurately but produces noisy vectors, indicating high sensitivity to pixel-level intensity changes with less smoothing/regularization.

Evaluation

For motion tracking in dynamic scenes (scenes with fast, unpredictable, and varied motion), Method A is generally more suitable, provided the missed small motions are not critical to the task.

- Accuracy: Method B is more accurate for detecting fine detail, but its noisy vectors can cause tracking errors, jitter, and false motion detections in a dynamic scene.
- Stability: Method A's stable vectors ensure consistent, reliable tracking of objects even as the scene changes rapidly, which is essential for maintaining a continuous track.
- Noise sensitivity: Dynamic scenes already introduce significant variability (camera shake, background clutter, multiple moving objects); Method B's noise would compound this problem, whereas Method A's stability filters out irrelevant fluctuations.
- Application requirements: Motion tracking applications (e.g., surveillance, vehicle tracking) prioritize consistent trajectory estimation over sensitivity to minor motions, favoring Method A.

However, if the application specifically requires detecting small or subtle movements (e.g., gesture recognition, micro-expression analysis), Method B would be preferred despite its noise, ideally combined with a post-processing filter to reduce jitter. In general dynamic-scene motion tracking, Method A's stability makes it the more practical choice.

4. Model Generalization and Overfitting

Model	Training Accuracy	Testing Accuracy
A	98%	70%
B	90%	88%

Comparison

- Model A: Very high training accuracy (98%) but a large drop in testing accuracy (70%), a gap of 28 percentage points. This large gap is a classic sign of overfitting — the model has memorized the training data (including noise) rather than learning generalizable patterns.
- Model B: Slightly lower training accuracy (90%) but testing accuracy remains close (88%), a gap of only 2 percentage points. This small gap indicates the model generalizes well to unseen data.

Generalization Ability and Overfitting Risk

- Model A has high overfitting risk: it fits the training set almost perfectly but fails to maintain that performance on new data, meaning it will likely perform poorly in real-world, unseen scenarios.
- Model B has low overfitting risk and strong generalization: its consistent performance across training and testing data suggests it has learned the underlying patterns rather than memorizing specific examples.

Evaluation and Selection

Model B is more reliable for real-world deployment. Although its training accuracy is slightly lower than Model A's, real-world performance is best predicted by testing (unseen data) accuracy and by the consistency between training and testing results — not by training accuracy alone. Model B's small train-test gap indicates it will perform reliably and consistently on new, unseen data in production, whereas Model A's large gap suggests unpredictable and degraded performance once deployed.

Recommendation: Select Model B, and if desired, further improve Model A using regularization, dropout, data augmentation, or early stopping to reduce its overfitting before considering it for deployment.

5. System Performance under Constraints

Model	Accuracy	Latency	Cost
A	88%	50 ms	Low
B	92%	80 ms	Moderate
C	95%	120 ms	High

Comparison

- Model A: Lowest accuracy (88%) but fastest latency (50 ms) and lowest cost — suitable when speed and budget are top priorities.
- Model B: Moderate accuracy (92%), moderate latency (80 ms), and moderate cost — a balanced middle option.
- Model C: Highest accuracy (95%) but highest latency (120 ms) and highest cost — best when accuracy is critical and resources/time are not constrained.

Evaluation

For a real-time application that must balance accuracy, latency, and cost, Model B provides the best overall balance.

- Accuracy vs. Latency: Model B improves accuracy over Model A by 4 percentage points while only increasing latency by 30 ms — a reasonable trade-off. Moving from B to C gains only 3 more accuracy points but adds another 40 ms of latency, a diminishing return.
- Cost efficiency: Model B's moderate cost is justified by its noticeably better accuracy than Model A, while avoiding the high cost of Model C.
- Real-time suitability: 80 ms latency is generally still acceptable for most real-time applications (well under typical 100–200 ms real-time thresholds), whereas Model C's 120 ms may be too slow for strict real-time constraints.

Justification

Model B offers the best cost-to-performance ratio: it delivers high accuracy (92%) close to Model C's, at a much lower latency and cost. Model A should only be chosen if latency and cost are the absolute top priorities and a lower accuracy (88%) is acceptable. Model C is justified only in scenarios where maximum accuracy is critical (e.g., safety-critical systems) and higher latency/cost can be tolerated. For general real-time deployment with practical constraints, Model B is the recommended choice.